

Linear regression and gradient descent

CSI 4106 - Fall 2026

Marcel Turcotte

Version: Sep 23, 2026 10:06

Preamble

Message of the Day



[The Invention of Hugo Larochelle](#), Amiral Ventures, 2026-09-14.

Hugo Larochelle is the scientific director of Mila and a Canadian computer scientist specializing in deep learning. Before joining Mila's leadership, he led Google Brain's Montréal research lab, which later became part of Google DeepMind, and served as an associate professor at the Université de Sherbrooke. He is also known for making advanced machine-learning courses freely available online.

I selected this profile for two reasons. First, it introduces Canada's three national AI institutes: Mila, Vector, and Amii. Second, it shows how Larochelle's education and early research shaped his career. It also connects directly to today's topic: Larochelle

describes linear regression as the point at which statistics began to make sense to him as a mathematically grounded form of machine learning.

- **Mila – Quebec Artificial Intelligence Institute (Montréal):** An academic research community with strengths in deep learning, generative AI, and socially responsible AI. Researchers associated with Mila include **Yoshua Bengio, Aaron Courville, Doina Precup, and Irina Rish.**
- **Vector Institute (Toronto):** An independent not-for-profit institute that advances AI research, develops talent, and supports adoption in Canadian industry and healthcare. Researchers associated with Vector include **Geoffrey Hinton, Richard Zemel, David Duvenaud, and Bo Wang.**
- **Amii (Alberta Machine Intelligence Institute, Edmonton):** An independent not-for-profit institute with longstanding strengths in reinforcement learning and machine learning. It also helps organizations adopt AI. Researchers associated with Amii include **Richard Sutton, Michael Bowling, Martha White, and Alona Fyshe.**

Learning Objectives

- **Recognize** regression as supervised learning with a real-valued label.
- **Define** a linear hypothesis and its mean squared error objective.
- **Distinguish** parameter space from hypothesis space, and explain how a parameter vector determines a hypothesis.
- **Explain** how the gradient and learning rate determine a gradient descent update.
- **Explain** why convexity matters when optimizing a linear regression model.
- **Compare** batch, stochastic, and mini-batch gradient descent.

In the previous lecture, we examined two distinct learning algorithms: k -nearest neighbors (KNN) and decision trees, each employing unique methods for model training and representation. KNN does not involve explicit learning; instead, the data itself constitutes the model. Conversely, decision trees utilize a greedy algorithm that begins with an empty tree and the full training dataset. The algorithm incrementally adds decision nodes, partitioning the parent dataset into subsets to achieve greater homogeneity (or purity) within each resulting classification compared to the parent node. This recursive process terminates when a node's data satisfies predefined stopping criteria, such as achieving a single class, reaching a minimum purity level, or attaining a maximum tree depth. Once the decision tree is constructed, the original data is no longer needed. This lecture aimed to demonstrate that supervised learning models can be represented in various ways, with each "learning" algorithm tailored to its specific model.

In today's lecture, we will explore a training algorithm that is applicable to a diverse range of models, including neural networks.

Linear Regression

Rationale

Linear regression is introduced to conveniently present a well-known training algorithm, **gradient descent**. Additionally, it serves as a foundation for introducing **logistic regression**—a classification algorithm—which further facilitates discussions on **artificial neural networks**.

- Linear Regression
 - Gradient Descent
 - Logistic Regression
 - Neural Networks

From 2 to **2 trillion** parameters!

The training algorithms for machine learning models can vary significantly depending on the model (e.g., decision trees, SVMs, etc.). In order to fit our schedule, we will concentrate on this specific sequence.

The concept of linear regression can be traced back to the early work of Sir Francis Galton in the late 19th century. Galton introduced the idea of “regression” in his 1886 paper, which focused on the relationship between the heights of parents and their children. He observed that children’s heights tended to regress towards the average, which led to the term “regression.”

Karl Pearson later formalized important ideas in correlation and regression. The method of least squares predates this work: Adrien-Marie Legendre published it in 1805, and Carl Friedrich Gauss developed it independently for astronomical data analysis.

See: Stanton (2001).

Supervised Learning - Regression

- The **training data** is a collection of **labelled** examples.
 - $\{(x_i, y_i)\}_{i=1}^N$
 - Each x_i is a **feature vector** with D dimensions.
 - $x_i^{(j)}$ is the value of the **feature** j of the example i , for $j \in 1 \dots D$ and $i \in 1 \dots N$.
 - The **label** y_i is a **real number**.
- **Problem:** Given the data set as input, create a **model** that can be used to predict the value of y for an unseen x .

Can you think of examples of regression tasks?

1. **House Price Prediction:**
 - **Application:** Estimating the market value of residential properties based on features such as location, size, number of bedrooms, age, and amenities.
2. **Stock Market Forecasting:**
 - **Application:** Predicting future prices of stocks or indices based on historical data, financial indicators, and economic variables.
3. **Weather Prediction:**
 - **Application:** Estimating future temperatures, rainfall, and other weather conditions using historical weather data and atmospheric variables.
4. **Sales Forecasting:**
 - **Application:** Predicting future sales volumes for products or services by analyzing past sales data, market trends, and seasonal patterns.
5. **Energy Consumption Prediction:**
 - **Application:** Forecasting future energy usage for households, industries, or cities based on historical consumption data, weather conditions, and economic factors.
6. **Medical Cost Estimation:**
 - **Application:** Predicting healthcare costs for patients based on their medical history, demographic information, and treatment plans.
7. **Traffic Flow Prediction:**
 - **Application:** Estimating future traffic volumes and congestion levels on roads and highways using historical traffic data and real-time sensor inputs.
8. **Customer Lifetime Value (CLV) Estimation:**
 - **Application:** Predicting the total revenue a business can expect from a customer over the duration of their relationship, based on purchasing behavior and demographic data.
9. **Economic Indicators Forecasting:**
 - **Application:** Predicting key economic indicators such as GDP growth, unemployment rates, and inflation using historical economic data and market trends.
10. **Demand Forecasting:**
 - **Application:** Estimating future demand for products or services in various industries like retail, manufacturing, and logistics to optimize inventory and supply chain management.
11. **Real Estate Valuation:**
 - **Application:** Assessing the market value of commercial properties like office buildings, malls, and industrial spaces based on location, size, and market conditions.
12. **Insurance Risk Assessment:**
 - **Application:** Predicting the risk associated with insuring individuals or properties, which helps in determining premium rates, based on historical claims data, and demographic factors.

13. Ad Click-Through Rate (CTR) Prediction:

- **Application:** Estimating the likelihood that a user will click on an online advertisement based on user behavior, ad characteristics, and contextual factors.

14. Loan Default Prediction:

- **Application:** Predicting the probability of a borrower defaulting on a loan based on credit history, income, loan amount, and other financial indicators.

Focusing on applications possibly running on a mobile device.

1. Battery Life Prediction:

- **Application:** Estimating remaining battery life based on usage patterns, running applications, and device settings.

2. Health and Fitness Tracking:

- **Application:** Predicting calorie burn, heart rate, or sleep quality based on user activity, biometrics, and historical health data.

3. Personal Finance Management:

- **Application:** Forecasting future expenses or savings based on spending habits, income patterns, and budget goals.

4. Weather Forecasting:

- **Application:** Providing personalized weather forecasts based on current location and historical weather data.

5. Traffic and Commute Time Estimation:

- **Application:** Predicting travel times and suggesting optimal routes based on historical traffic data, real-time conditions, and user behavior.

6. Image and Video Quality Enhancement:

- **Application:** Adjusting image or video quality settings (e.g., brightness, contrast) based on lighting conditions and user preferences.

7. Fitness Goal Achievement:

- **Application:** Estimating the time needed to achieve fitness goals such as weight loss or muscle gain based on user activity and dietary input.

8. Mobile Device Performance Optimization:

- **Application:** Predicting the optimal settings for device performance and battery life based on usage patterns and app activity.

Old Faithful Eruptions

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

WOLFRAM_CSV = "https://raw.githubusercontent.com/turcotte/csi4106-f26/refs/heads/main/data/old_faithful.csv"
faithful_df = pd.read_csv(WOLFRAM_CSV)

# Renaming the columns
```

```
faithful_df = faithful_df.rename(  
    columns={"Duration": "eruptions", "WaitingTime": "waiting"}  
)  
print(faithful_df.shape)  
faithful_df.head(6)
```

(272, 2)

Attribution: Wolfram Research, "Sample Data: Old Faithful Eruptions" from the Wolfram Data Repository (2016) [doi: 10.24097/wolfram.50727.data](https://doi.org/10.24097/wolfram.50727.data)

The dataset used in this presentation was sourced from the Wolfram Research Data Repository, with its initial publication detailed in Azzalini and Bowman (1990).

Old Faithful Geyser

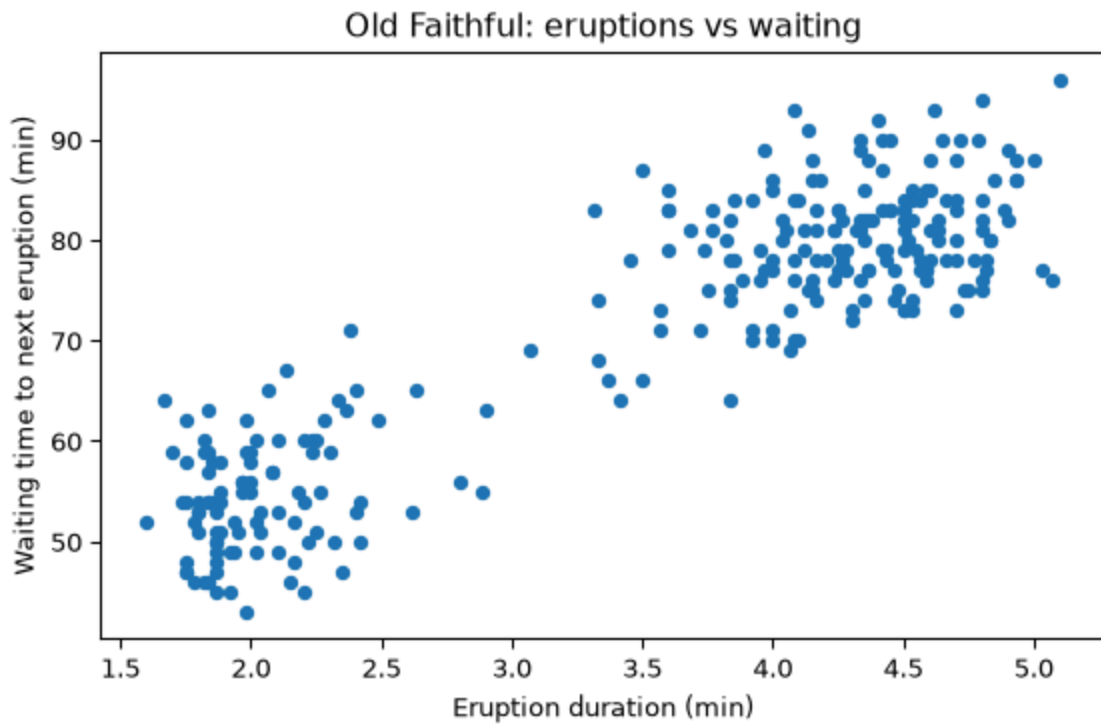
<https://youtu.be/Qxf3xzirBrs>

Attribution: Yellowstone National Park Trips

Old Faithful, situated in Yellowstone National Park, is renowned as the world's most famous geyser. It can reach eruption heights of up to 140 feet. Notably, its eruption intervals range between 60 and 110 minutes, contingent upon the duration of the preceding eruption.

Quick Visualization

```
In [3]: plt.figure(figsize=(6,4))  
plt.scatter(faithful_df["eruptions"], faithful_df["waiting"], s=20)  
plt.xlabel("Eruption duration (min)")  
plt.ylabel("Waiting time to next eruption (min)")  
plt.title("Old Faithful: eruptions vs waiting")  
plt.tight_layout()  
plt.show()
```



The duration of the current eruption appears to have a linear relationship with the subsequent wait time: shorter eruption durations tend to precede shorter wait times, while longer eruption durations are associated with longer wait times.

Problem

- Predict the **waiting time until the next eruption (min)**, y , based on the **duration of the current eruption (min)**, x .

Selecting a problem characterized by a single attribute allows us to better focus our discussion and enhance the clarity of visualization.

<https://youtu.be/qxo8p8PtFeA?si=Buy1DF-T1qPsVE2S>

Linear Regression

A **linear model** assumes that the predicted value, $\hat{y}_i$, can be expressed as a **linear combination** of the feature values, $x_i^{(j)}$:

$$\hat{y}_i = \theta_0 + \theta_1 x_i^{(1)} + \theta_2 x_i^{(2)} + \dots + \theta_D x_i^{(D)}$$

...

Here, θ_j is the j th parameter of the (linear) **model**, with θ_0 being the **bias** term/parameter, and $\theta_1 \dots \theta_D$ being the **feature weights**.

In my presentations, I use $\hat{y}_i$ and $h(x_i)$ synonymously.

In statistical contexts, the notation $\hat{y}_i$ denotes the predicted or estimated value of y_i under a given model.

Conversely, in machine learning, the notation $h(x_i)$ is used, where h represents the hypothesis function or model applied to the input data x_i . The hypothesis function h is derived from a predefined hypothesis space, which encompasses the set of all possible models that can be used to map input data to predicted outcomes.

The parameter θ_0 is called the **bias term** (also "intercept") because:

- It **shifts the prediction independently of the inputs**.
- Geometrically, it moves the regression hyperplane up or down (or left/right in classification), so the model is not forced to pass through the origin.
- In machine learning terms, it acts like a **constant offset**, compensating for systematic effects not explained by the features.

So it's called "bias" because it introduces a fixed baseline to which the contributions of the other parameters are added.

In a machine learning model, the **parameters** are the **weights** and the **biases**.

Definition

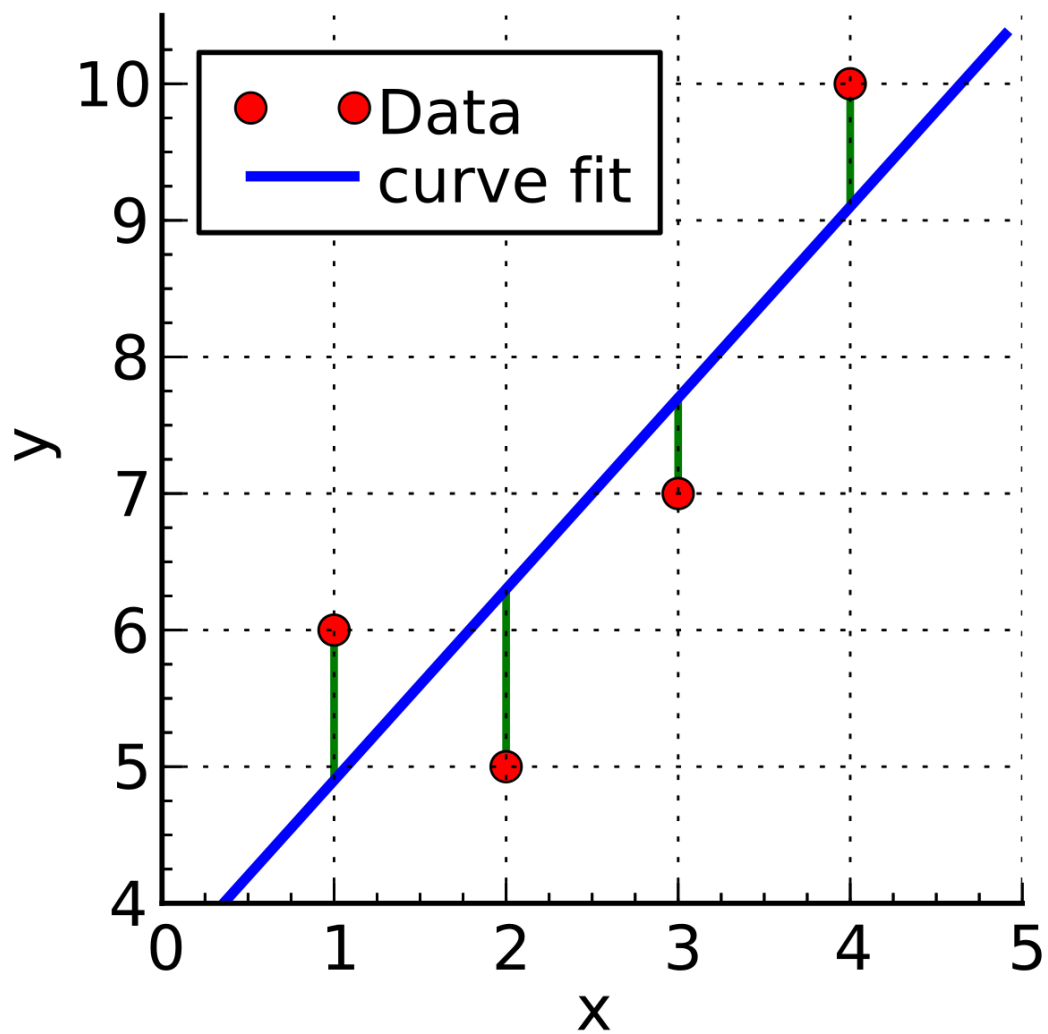
Problem: find values for all the model parameters so that the model **best fits** the training data.

- The **Mean Squared Error (MSE)** is a common objective for regression problems.

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N [h_{\theta}(x_i) - y_i]^2$$

The root mean squared error (RMSE) is the square root of MSE. Because the square-root function is strictly increasing, MSE and RMSE have the same minimizing parameter values. We optimize MSE because its derivative is simpler, while RMSE remains useful for reporting error in the same units as the target.

Minimizing MSE



Attribution: [Krishnavedala](#), CC BY-SA 3.0, via Wikimedia Commons

Learning

```
In [4]: from sklearn.linear_model import LinearRegression, SGDRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Prepare data
faithful_X = faithful_df[["eruptions"]].values # shape (n_samples, 1)
faithful_y = faithful_df["waiting"].values    # shape (n_samples,)

faithful_X_train, faithful_X_test, faithful_y_train, faithful_y_test = train_test_split(
    faithful_X, faithful_y, test_size=0.2, random_state=42
)

# Fit via SGDRegressor - linear model via gradient descent
sgd = SGDRegressor(
    loss="squared_error",
    penalty=None,
```

```

    learning_rate="constant",
    eta0=0.01, # Initial learning rate (alpha in our notation).
    max_iter=2000,
    tol=None,
    random_state=42
)

sgd.fit(faithful_X_train, faithful_y_train)

print("Learned parameters:")
print(f"    intercept = {sgd.intercept_[0]:.3f}")
print(f"    slope      = {sgd.coef_[0]:.3f}")

faithful_y_pred = sgd.predict(faithful_X_test)
print(f"Test MSE = {mean_squared_error(faithful_y_test, faithful_y_pred):.2f}")
print(f"Test R² = {r2_score(faithful_y_test, faithful_y_pred):.3f}")

```

```

Learned parameters:
    intercept = 32.910
    slope      = 10.503
Test MSE = 43.02
Test R² = 0.671

```

Visualization

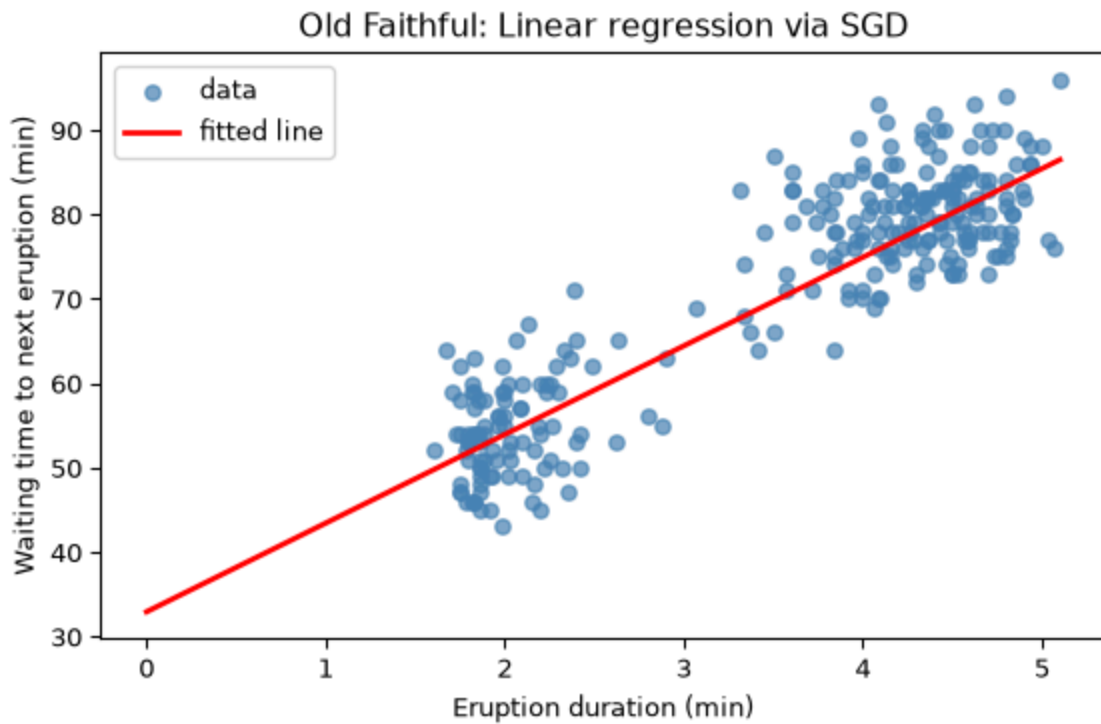
```

In [5]: # Scatter the data
plt.figure(figsize=(6,4))
plt.scatter(faithful_X, faithful_y, color="steelblue", s=30, alpha=0.7, label="data")

# Plot the fitted line
x_line = np.linspace(0, faithful_X.max(), 100).reshape(-1, 1)
y_line = sgd.predict(x_line)
plt.plot(x_line, y_line, color="red", linewidth=2, label="fitted line")

plt.xlabel("Eruption duration (min)")
plt.ylabel("Waiting time to next eruption (min)")
plt.title("Old Faithful: Linear regression via SGD")
plt.legend()
plt.tight_layout()
plt.show()

```



The horizontal range extends to $x = 0$ so that the intercept, $\theta_0 = 32.910$, is visible.

Characteristics

A typical **learning algorithm** comprises the following components:

1. A **model**, often consisting of a set of **parameters** whose values will be learned.
2. An **objective function** that measures prediction error on the training data.
 - The **Mean Squared Error** is a common objective for regression problems.

$$\frac{1}{N} \sum_{i=1}^N [h_{\theta}(x_i) - y_i]^2$$

3. **Optimization** algorithm

Optimization

Until a termination criterion is met[1]:

- **Evaluate** the loss function, comparing $h(x_i)$ to y_i .
- **Make small changes to the parameters**, in a way that reduces the value of the loss function.

Remarks

- It is important to separate the **optimization algorithm** from the **problem** it addresses.
- For **linear regression**, an exact analytical solution exists, but it presents certain limitations.
- **Gradient descent** serves as a general algorithm applicable not only to linear regression, but also to logistic regression, deep learning, t-SNE (t-distributed Stochastic Neighbor Embedding), among various other problems.
- There exists a diverse range of optimization algorithms that **do not rely on gradient-based methods**.

Optimization — single feature

- **Model** (hypothesis):

$$h(x_i; \theta) = \theta_0 + \theta_1 x_i^{(1)}$$

- **Loss/cost function:**

$$J(\theta_0, \theta_1) = \frac{1}{N} \sum_{i=1}^N [h(x_i; \theta) - y_i]^2$$

Goal: find θ_0, θ_1 that minimize J , by iteratively updating the parameters.

This slide establishes the distinction between the hypothesis space and the parameter space.

The parameter space is $\Theta = \mathbb{R}^2$. Each point $\theta = (\theta_0, \theta_1)$ selects one hypothesis, $h_\theta(x) = \theta_0 + \theta_1 x$, from the hypothesis space $\mathcal{H} = \{h_\theta \mid \theta \in \Theta\}$. Once training finishes, the selected parameters remain fixed and the model maps a new input x_{new} to a prediction.

For a fixed dataset D , the objective J_D maps each parameter pair to a non-negative MSE: $J_D : \Theta \rightarrow \mathbb{R}_{\geq 0}$. Changing θ selects a different hypothesis, while the dataset remains fixed.

Gradient descent operates in parameter space. It adjusts θ to reduce $J_D(\theta)$.

The notation $h(x_i; \theta)$ indicates that the value of the function h (of the model) depends on the input example x_i as well as the parameters θ . The semicolon is used to semantically differentiate these two sets of values. This convention comes from the field of statistics. In machine learning, we also use the notation $h_\theta(x_i)$, which can be considered more appropriate. In this context, we refer to an indexed family of functions, where θ specifically determines which function h_θ should be used. This notation is read as follows: "the function h , parameterized by θ , applied to the input x_i ."

Hypotheses and Parameter Space

[1] E.g. the value of the **loss function no longer decreases** or the **maximum number of iterations**.

```
In [6]: faithful_x = faithful_X.ravel().astype(float)
faithful_y = faithful_y.astype(float)

# The objective is defined on the complete, fixed dataset.
def linear_mse(theta_0, theta_1):
    predictions = theta_0 + theta_1 * faithful_x
    return np.mean((predictions - faithful_y) ** 2)

# Exact least-squares minimizer for the complete dataset.
full_data_model = LinearRegression().fit(faithful_X, faithful_y)
theta_star = np.array([
    full_data_model.intercept_,
    full_data_model.coef_[0],
])

# An illustrative sequence of parameter choices ending at the minimizer.
theta_path = np.array([
    (0.0, 0.0),
    (10.0, 2.0),
    (20.0, 4.0),
    (30.0, 7.0),
    theta_star,
])

path_costs = np.array([
    linear_mse(theta_0, theta_1)
    for theta_0, theta_1 in theta_path
])

# Evaluate the same objective throughout parameter space.
theta_0_values = np.linspace(0, 100, 200)
theta_1_values = np.linspace(0, 20, 200)
theta_0_grid, theta_1_grid = np.meshgrid(theta_0_values, theta_1_values)
cost_grid = np.empty_like(theta_0_grid, dtype=float)
for row in range(theta_0_grid.shape[0]):
    for column in range(theta_1_grid.shape[1]):
        cost_grid[row, column] = linear_mse(
            theta_0_grid[row, column],
            theta_1_grid[row, column],
        )

# Left: selected hypotheses. Right: their objective values in parameter space
fig = plt.figure(figsize=(12, 4.8))

ax1 = fig.add_subplot(1, 2, 1)
input_order = np.argsort(faithful_x)
ax1.scatter(faithful_x, faithful_y, s=18, alpha=0.75, label="data")
ax1.axhline(faithful_y.mean(), color="gray", ls="--", lw=1, label=r"$\bar{y}$")
for index, (theta_0, theta_1) in enumerate(theta_path):
    hypothesis_values = theta_0 + theta_1 * faithful_x
```

```

ax1.plot(
    faithful_x[input_order], hypothesis_values[input_order],
    lw=2 if index == len(theta_path) - 1 else 1.2,
    alpha=1.0 if index == len(theta_path) - 1 else 0.85,
    label=fr"$\theta_0={\theta_0:.3f}, \ \theta_1={\theta_1:.3f}$",
)
ax1.set_xlabel("Eruption duration (min)")
ax1.set_ylabel("Waiting time to next eruption (min)")
ax1.set_title("Selected hypotheses in input-output space")
ax1.legend(fontsize=8, loc="best")

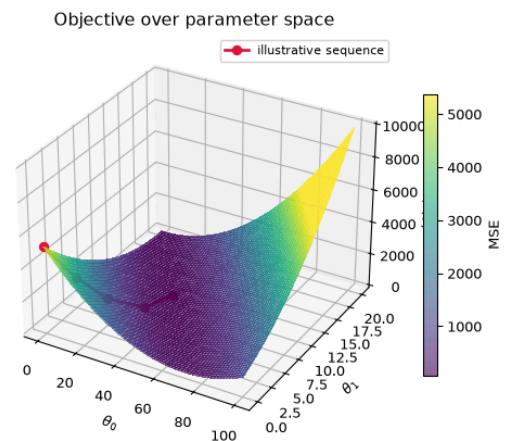
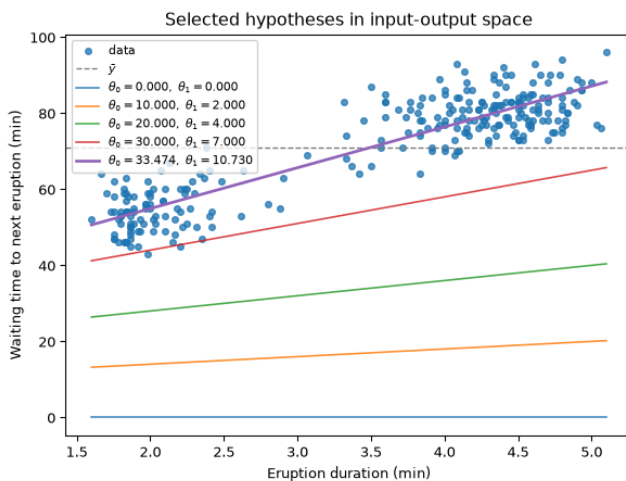
ax2 = fig.add_subplot(1, 2, 2, projection='3d')
vmin = np.percentile(cost_grid, 5)
vmax = np.percentile(cost_grid, 95)

surf = ax2.plot_surface(
    theta_0_grid, theta_1_grid, cost_grid, rstride=3, cstride=3,
    cmap="viridis", linewidth=0, antialiased=False,
    vmin=vmin, vmax=vmax, alpha=0.6,
)
ax2.plot(
    theta_path[:, 0], theta_path[:, 1], path_costs,
    color='crimson', marker='o', lw=2,
    label="illustrative sequence",
)

ax2.set_xlabel(r'$\theta_0$')
ax2.set_ylabel(r'$\theta_1$')
ax2.set_zlabel(r'$J(\theta)$ (MSE)')
ax2.set_title(r"Objective over parameter space")
ax2.legend(loc="best", fontsize=8)
ax2.view_init(elev=30, azim=-60)

fig.colorbar(surf, ax=ax2, shrink=0.7, pad=0.05, label="MSE")
plt.tight_layout()
plt.show()

```



In this example, we use the Old Faithful geyser eruption dataset.

The figure on the left shows selected members of the hypothesis space in the input-output plane. Each parameter pair (θ_0, θ_1) defines one complete function, $h_{\theta}(x) = \theta_0 + \theta_1 x$, shown as a line.

The figure on the right depicts the parameter space, $\Theta = \mathbb{R}^2$. Each horizontal location is a parameter pair, and the surface height is the MSE incurred by its corresponding hypothesis on the complete, fixed dataset.

The parameter sequence is illustrative rather than an actual gradient descent trajectory. Moving to each new point in parameter space selects a new hypothesis on the left and lowers $J(\theta)$ on the right. The final pair is the full-data least-squares minimizer.

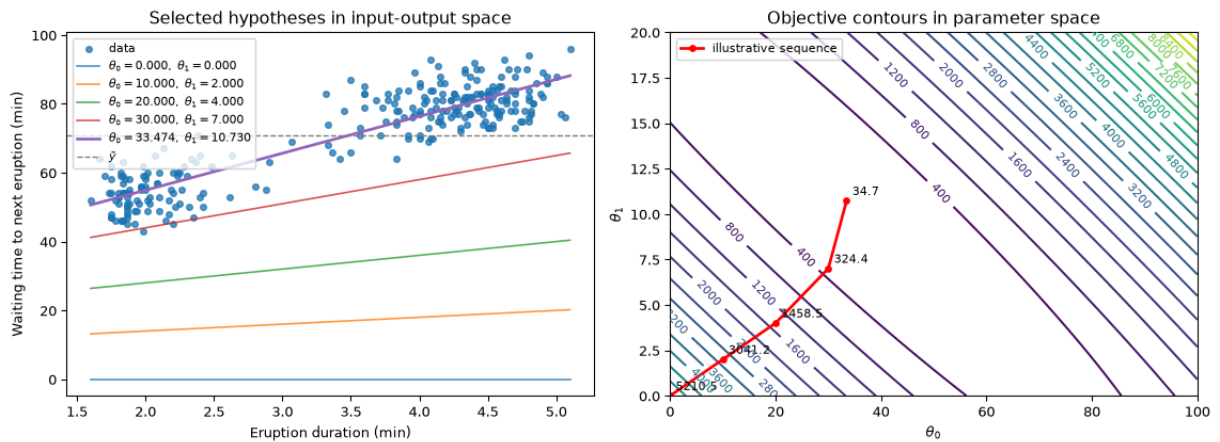
Hypotheses and Parameter Space

```
In [7]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.5))

# Reuse the same dataset, objective, grid, and parameter sequence.
ax1.scatter(faithful_x, faithful_y, s=18, alpha=0.75, label="data")
for index, (theta_0, theta_1) in enumerate(theta_path):
    hypothesis_values = theta_0 + theta_1 * faithful_x
    ax1.plot(
        faithful_x[input_order], hypothesis_values[input_order],
        lw=2 if index == len(theta_path) - 1 else 1.2,
        alpha=1.0 if index == len(theta_path) - 1 else 0.8,
        label=f"$\theta_0={theta_0:.3f}, \theta_1={theta_1:.3f}$",
    )
ax1.axhline(faithful_y.mean(), color="gray", ls="--", lw=1, label=r"$\bar{y}$")
ax1.set_xlabel("Eruption duration (min)")
ax1.set_ylabel("Waiting time to next eruption (min)")
ax1.set_title("Selected hypotheses in input-output space")
ax1.legend(fontsize=8, loc="best")

# Show the same objective over parameter space as a contour map.
cs = ax2.contour(theta_0_grid, theta_1_grid, cost_grid, levels=30)
ax2.clabel(cs, inline=True, fontsize=8)
ax2.plot(
    theta_path[:, 0], theta_path[:, 1], 'r-o', lw=2, ms=4,
    label="illustrative sequence",
)
for theta_0, theta_1, cost in zip(
    theta_path[:, 0], theta_path[:, 1], path_costs
):
    ax2.annotate(
        f"{cost:.1f}", (theta_0, theta_1),
        textcoords="offset points", xytext=(4, 4), fontsize=8,
    )
ax2.set_xlim(0, 100)
ax2.set_ylim(0, 20)
ax2.set_xlabel(r'$\theta_0$')
ax2.set_ylabel(r'$\theta_1$')
ax2.set_title("Objective contours in parameter space")
ax2.legend(loc="best", fontsize=8)
```

```
plt.tight_layout()
plt.show()
```



These figures show the same mapping using objective contours rather than a three-dimensional surface.

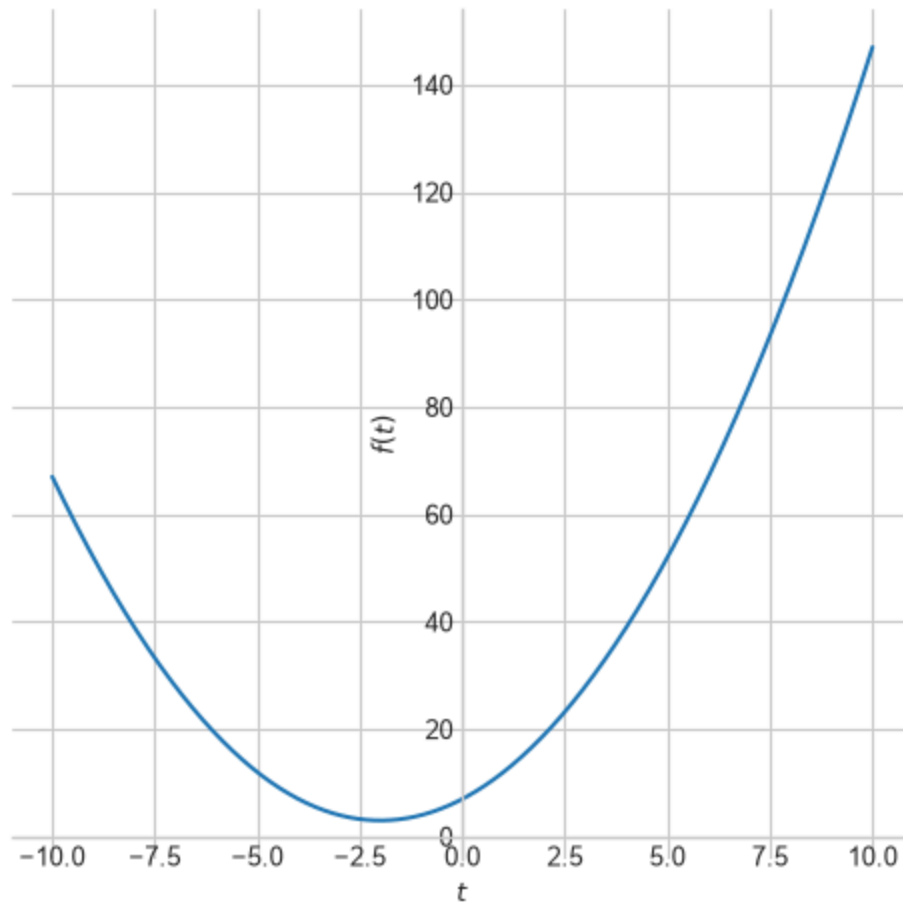
Derivative

Derivative

```
In [8]: import sympy as sp
from matplotlib import style

style.use("seaborn-v0_8-whitegrid")
t = sp.symbols("t")
quadratic_expression = t**2 + 4*t + 7
sp.plot(quadratic_expression, size=(5, 5))

sp.plot(quadratic_expression, size=(5, 5))
```



- We will start with a **single-variable function**.
- Think of it as an objective that we want to minimize.
- We use t to distinguish the function's input from the features in our training examples.

Source code

```
In [10]: quadratic_derivative = sp.diff(quadratic_expression, t)
quadratic_function = sp.lambdify(t, quadratic_expression, "numpy")
derivative_function = sp.lambdify(t, quadratic_derivative, "numpy")
t_values = np.linspace(-5, 2, 400)

def plot_quadratic_and_derivative(shade=None, objective=False):
    function_values = quadratic_function(t_values)
    derivative_values = derivative_function(t_values)
    function_label = r"$J$" if objective else r"$f(t) = t^2 + 4t + 7$"
    derivative_label = (
        r"$\frac{\partial J}{\partial \theta_j}$"
        if objective else r"$f'(t) = 2t + 4$"
    )

    plt.plot(t_values, function_values, label=function_label, color="blue")
    plt.plot(t_values, derivative_values, label=derivative_label, color="red")
    if shade == "positive":
```

```

plt.fill_between(
    t_values, derivative_values,
    where=(derivative_values > 0), color="red", alpha=0.3,
)
elif shade == "negative":
    plt.fill_between(
        t_values, derivative_values,
        where=(derivative_values < 0), color="red", alpha=0.3,
    )

plt.axhline(0, color="black", linewidth=1)
plt.axvline(0, color="black", linewidth=1)
plt.xlabel(r"$\theta_j$" if objective else "$t$")
plt.ylabel("$J$" if objective else "$f(t)$")
plt.legend()
plt.grid(True)
plt.show()

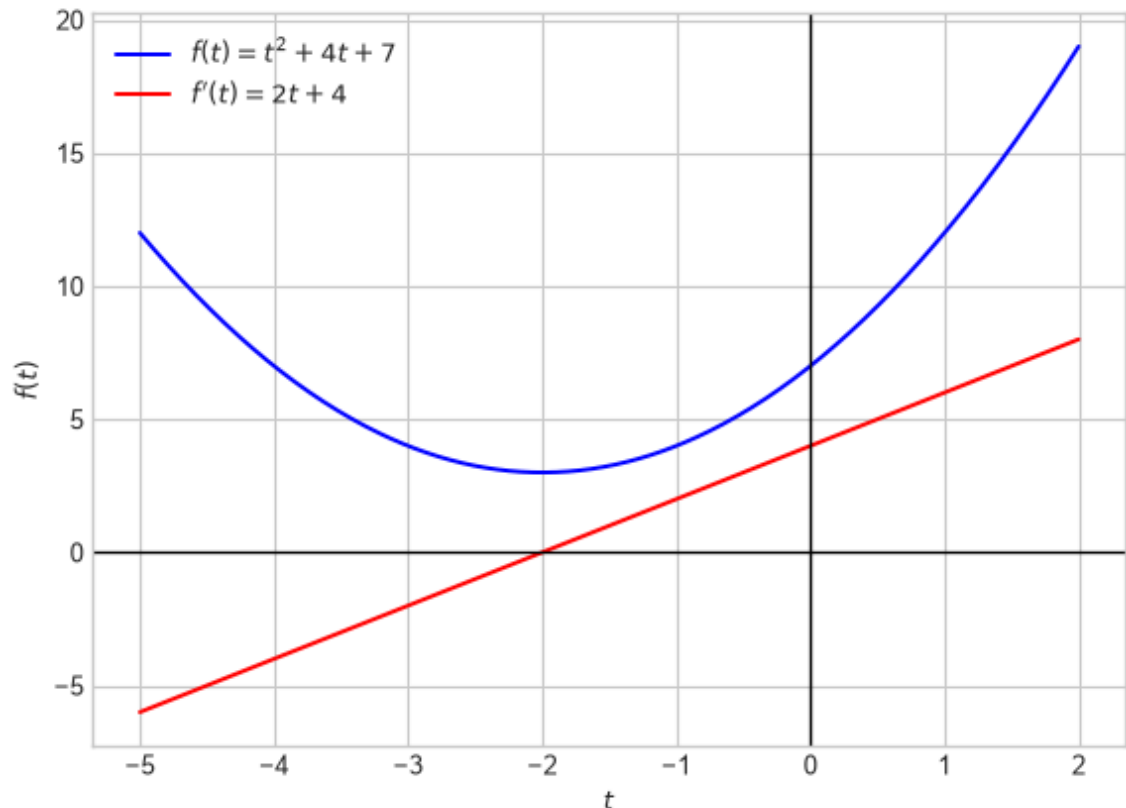
```

The previous slide uses [SymPy](#), a library for **symbolic mathematics**.

Derivative

In [11]: `plot_quadratic_and_derivative()`

`plot_quadratic_and_derivative()`



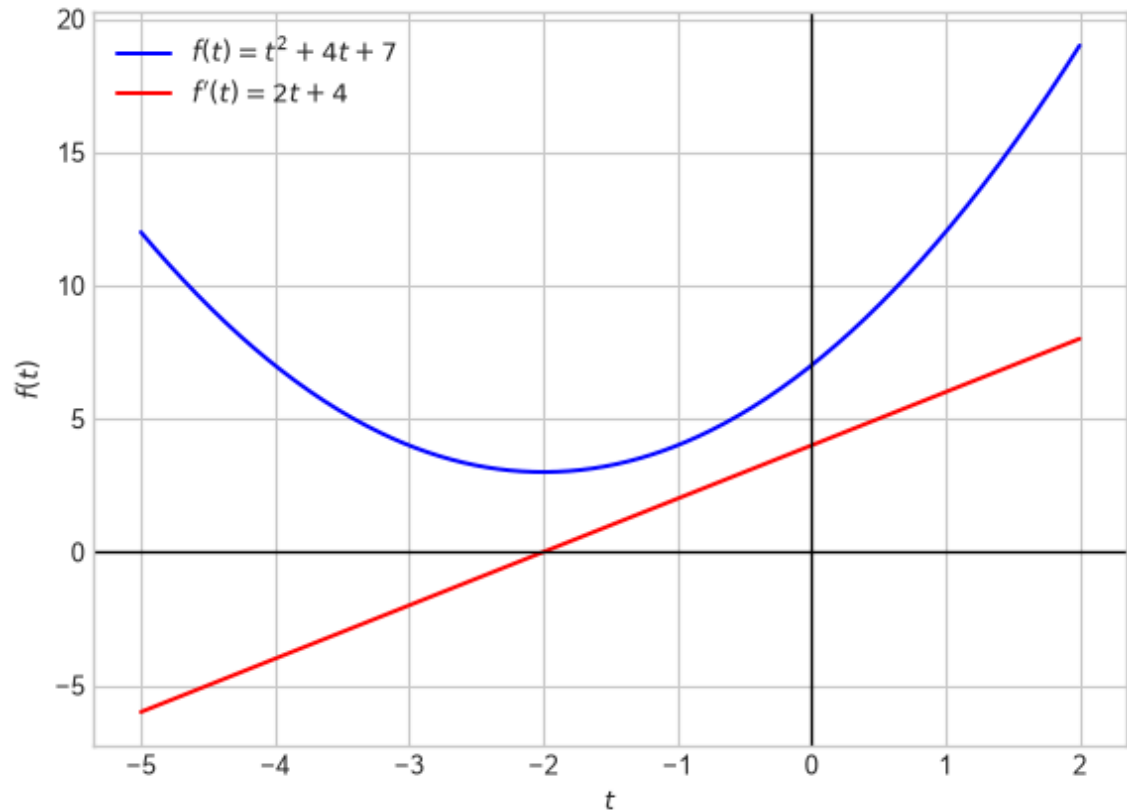
- The graph of the **derivative**, $f'(t)$, is depicted in **red**.
- The derivative describes how a small change in the input changes the output.

- At $t = -2$, the derivative equals 0.
- This point is the minimum of the function.

Near $t = -2$, small changes in t have little effect on $f(t)$.

Derivative

`plot_quadratic_and_derivative()`

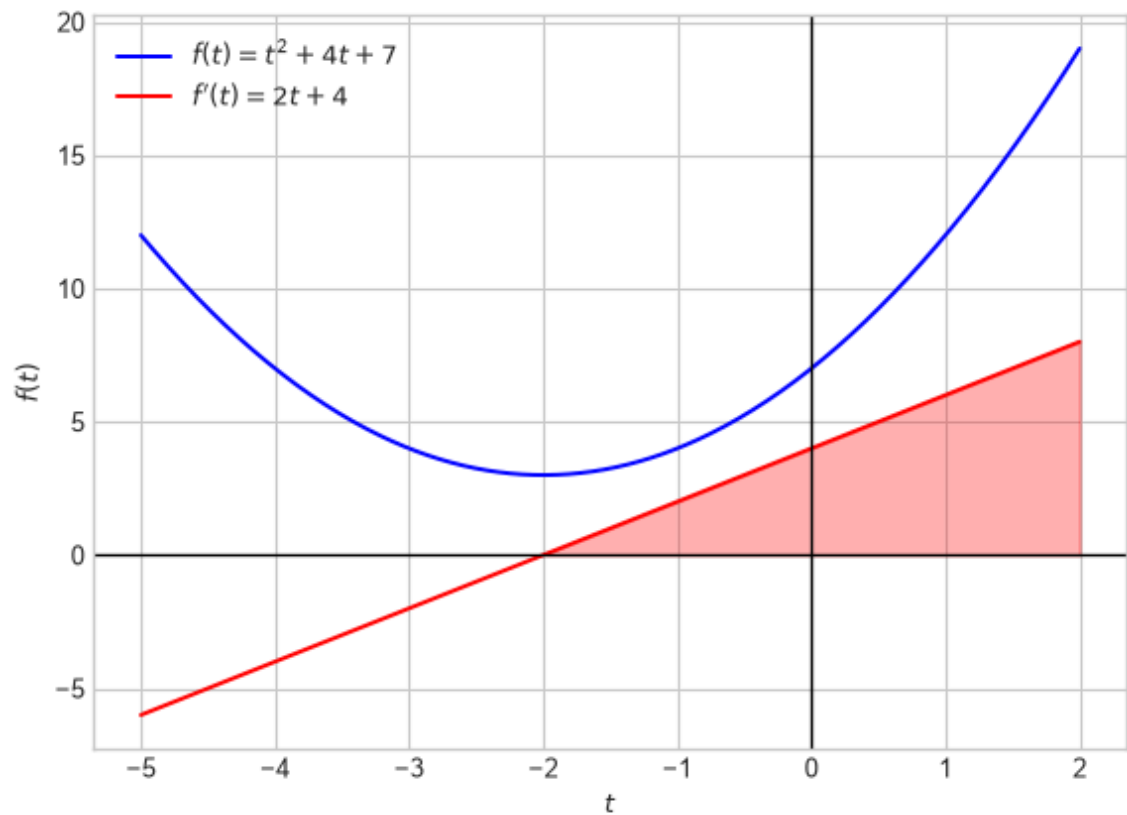


- At a specific point, the derivative gives the **slope of the tangent line** to the graph.
- At $t = -2$, the tangent line has slope 0.

Derivative

```
In [14]: plot_quadratic_and_derivative(shade="positive")

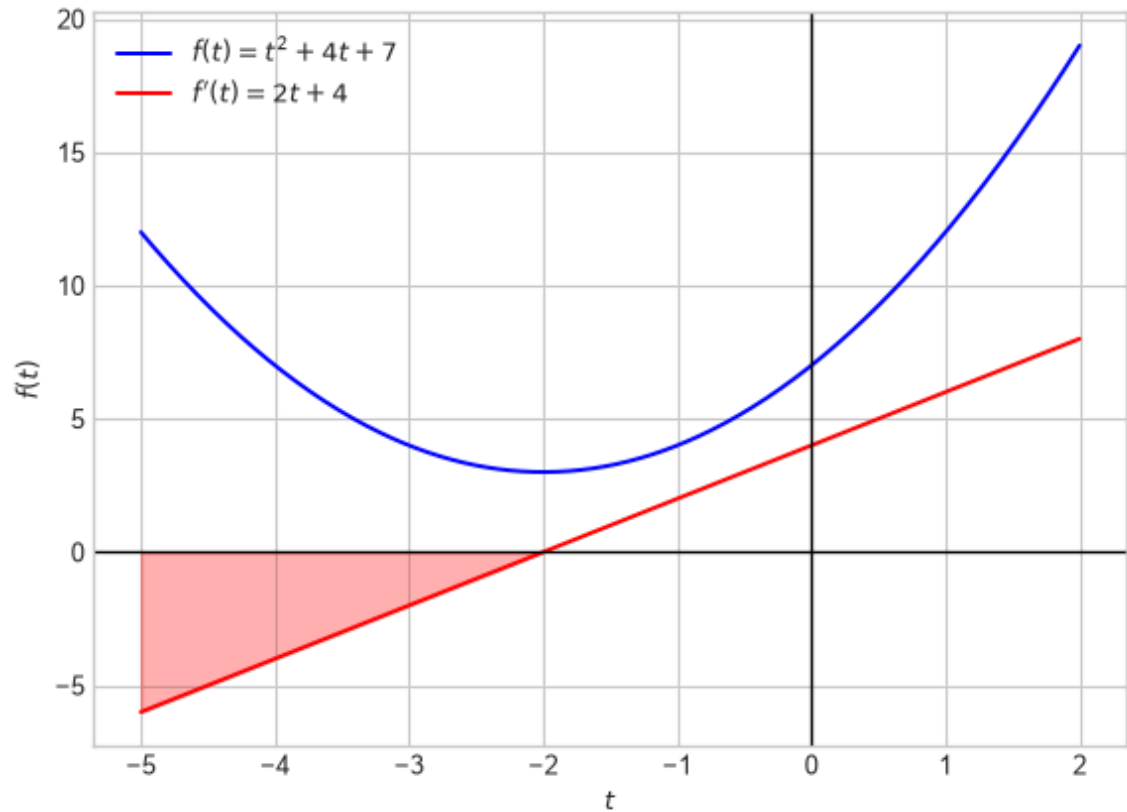
plot_quadratic_and_derivative(shade="positive")
```



- A **positive derivative** means that increasing the input increases the output.
- Its magnitude indicates how rapidly the output changes.

Derivative

```
In [16]: plot_quadratic_and_derivative(shade="negative")  
  
plot_quadratic_and_derivative(shade="negative")
```



- A **negative derivative** means that increasing the input decreases the output.
- Its magnitude indicates how rapidly the output changes.

Source code

```
In [18]: plot_quadratic_and_derivative(shade="positive")
```

Gradient Descent

Gradient Descent — Single Feature

- **Model** (hypothesis):

$$h(x_i; \theta) = \theta_0 + \theta_1 x_i^{(1)}$$

- **Loss/cost function:**

$$J(\theta_0, \theta_1) = \frac{1}{N} \sum_{i=1}^N [h(x_i; \theta) - y_i]^2$$

Gradient descent - intuition

<https://youtu.be/i62czvwDlsw>

Gradient Descent - Step-by-Step

<https://www.youtube.com/watch?v=sDv4f4s2SB8>

Gradient descent - single feature

- **Initialization:** Set θ_0 and θ_1 to random values or zeros.
- **Loop:**
 - repeat until convergence:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1), \text{ for } j = 0 \text{ and } j = 1$$

- α is the **learning rate**, which controls the size of each step.
- $\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$ is the **partial derivative** with respect to θ_j .

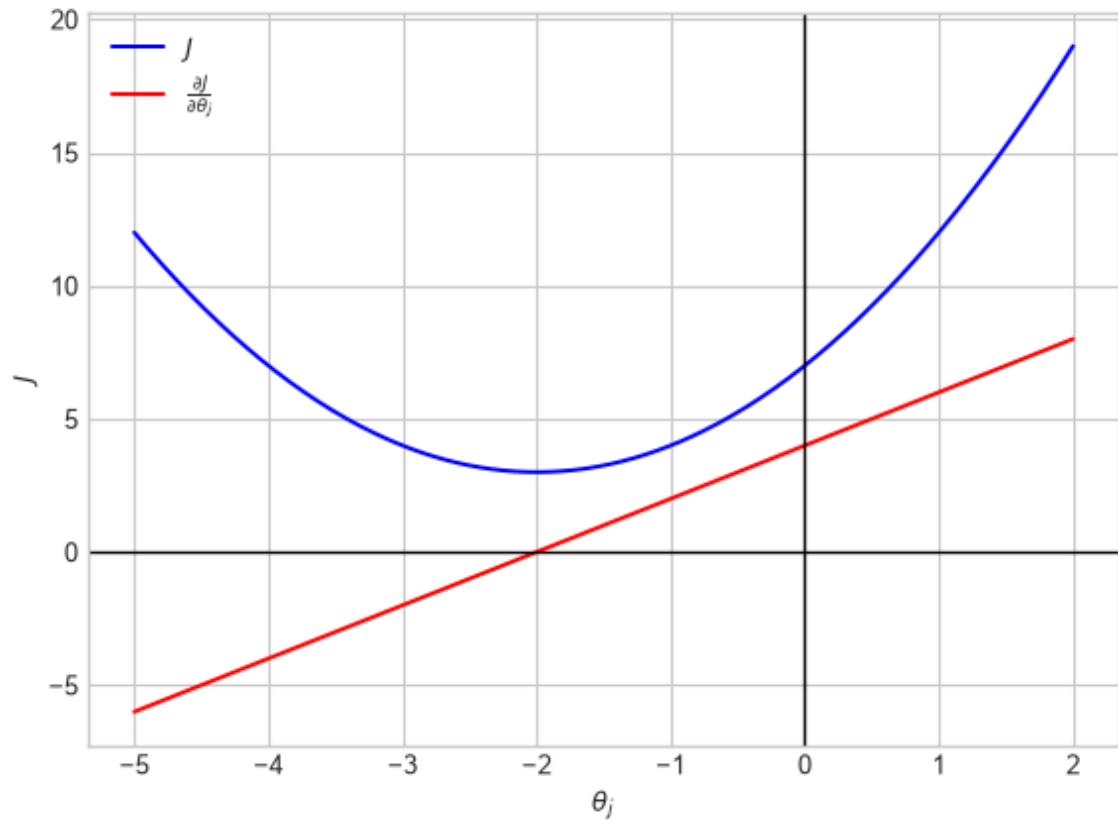
A **partial derivative** represents the rate of change of a multivariable function **with respect to one of its variables**, while **keeping the other variables constant**.

For the algorithm to be mathematically sound, all the θ_j must be updated **simultaneously**.

One-dimensional update

```
In [19]: plot_quadratic_and_derivative(objective=True)

plot_quadratic_and_derivative(objective=True)
```

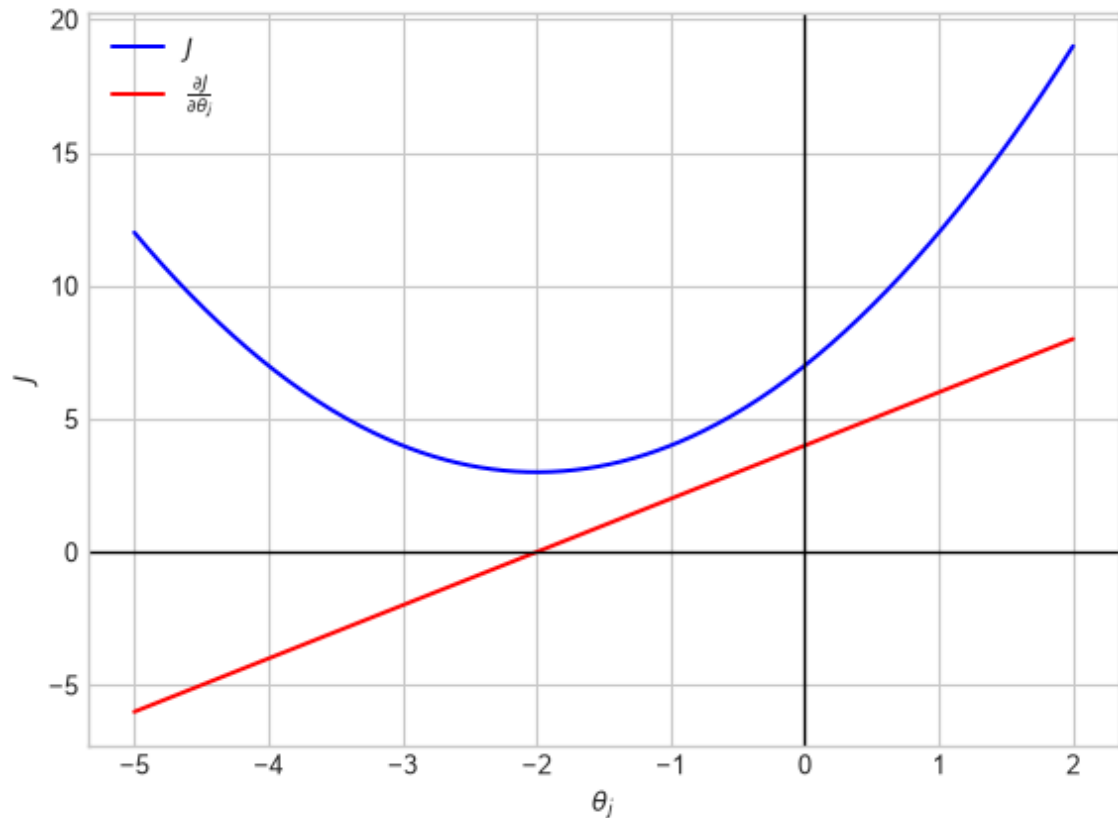


- When $\theta_j \in (-\infty, -2)$, $\frac{\partial}{\partial \theta_j} J(\theta)$ is **negative**.
- Therefore, $-\alpha \frac{\partial}{\partial \theta_j} J(\theta)$ is **positive**.
- Accordingly, the value of θ_j is **increased**.

Updating rule: $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$, for $j = 0$ and $j = 1$.

One-dimensional update

`plot_quadratic_and_derivative(objective=True)`



- When $\theta_j \in (-2, \infty)$, $\frac{\partial}{\partial \theta_j} J(\theta)$ is **positive**.
- Therefore, $-\alpha \frac{\partial}{\partial \theta_j} J(\theta)$ is **negative**.
- Accordingly, the value of θ_j is **decreased**.

Updating rule: $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$, for $j = 0$ and $j = 1$.

Partial derivatives

Given

$$J(\theta_0, \theta_1) = \frac{1}{N} \sum_{i=1}^N [h_{\theta}(x_i) - y_i]^2 = \frac{1}{N} \sum_{i=1}^N [\theta_0 + \theta_1 x_i - y_i]^2$$

...

We have

$$\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{2}{N} \sum_{i=1}^N [\theta_0 + \theta_1 x_i - y_i]$$

...

and

$$\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{2}{N} \sum_{i=1}^N x_i [\theta_0 + \theta_1 x_i - y_i]$$

Partial derivative (SymPy)

```
In [22]: from IPython.display import Math, display

# Define the index, parameters, and indexed data.
i, N = sp.symbols("i N", integer=True, positive=True)
theta_0, theta_1 = sp.symbols("theta_0 theta_1")
x_data = sp.IndexedBase("x")
y_data = sp.IndexedBase("y")
h_i = theta_0 + theta_1 * x_data[i]

print("Hypothesis function:")
display(Math(r"h\_theta(x_i) = " + sp.latex(h_i)))
```

Hypothesis function:

Partial derivative (SymPy)

```
In [23]: # Define the mean squared error, summing over example index i.
J_symbolic = sp.Sum((h_i - y_data[i])**2, (i, 1, N)) / N

print("Loss function:")
display(Math(r"J = " + sp.latex(J_symbolic)))
```

Loss function:

Partial derivative (SymPy)

```
In [24]: # Calculate the partial derivative with respect to theta_0

partial_derivative_theta_0 = sp.diff(J_symbolic, theta_0)

print("Partial derivative with respect to theta_0:")

display(Math(sp.latex(partial_derivative_theta_0)))
```

Partial derivative with respect to theta_0:

Partial derivative (SymPy)

```
In [25]: # Calculate the partial derivative with respect to theta_1

partial_derivative_theta_1 = sp.diff(J_symbolic, theta_1)

print("\nPartial derivative with respect to theta_1:")
```

```
display(Math(sp.latex(partial_derivative_theta_1)))
```

Partial derivative with respect to θ_1 :

Multivariate linear regression

$$h_{\theta}(x_i) = \sum_{j=0}^D \theta_j x_i^{(j)} = \theta^{\top} x_i, \quad x_i^{(0)} = 1$$

$x_i^{(j)}$ = value of feature j in example i

D = the number of features

Gradient descent - multivariate

The new **loss function** is

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N [h_{\theta}(x_i) - y_i]^2$$

Its **partial derivative**:

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{2}{N} \sum_{i=1}^N x_i^{(j)} [\theta^{\top} x_i - y_i]$$

Here, θ and x_i are vectors. Their dot product, $\theta^{\top} x_i$, and the target y_i are scalars.

Gradient vector

The vector containing the partial derivative of J (with respect to θ_j , for $j \in \{0, 1 \dots D\}$) is called the **gradient vector**.

$$\nabla_{\theta} J(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} J(\theta) \\ \frac{\partial}{\partial \theta_1} J(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_D} J(\theta) \end{pmatrix}$$

...

- This vector gives the direction of the **steepest ascent**.
- It gives its name to the **gradient descent** algorithm:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta} J(\theta^{(t)})$$

Gradient descent - multivariate

The gradient descent algorithm becomes:

Repeat until convergence:

$$\left\{ \begin{array}{l} \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1, \dots, \theta_D) \\ \text{for } j \in \{0, \dots, D\} \text{ (update simultaneously)} \end{array} \right\}$$

Gradient descent - multivariate

Repeat until convergence:

$$\left\{ \begin{array}{l} \theta_0 := \theta_0 - \alpha \frac{2}{N} \sum_{i=1}^N x_i^{(0)} [h_{\theta}(x_i) - y_i] \\ \theta_1 := \theta_1 - \alpha \frac{2}{N} \sum_{i=1}^N x_i^{(1)} [h_{\theta}(x_i) - y_i] \\ \theta_2 := \theta_2 - \alpha \frac{2}{N} \sum_{i=1}^N x_i^{(2)} [h_{\theta}(x_i) - y_i] \\ \dots \end{array} \right\}$$

For every example, $x_i^{(0)} = 1$.

Assumptions

What were our **assumptions**?

...

- The (objective/loss) function is **differentiable**.

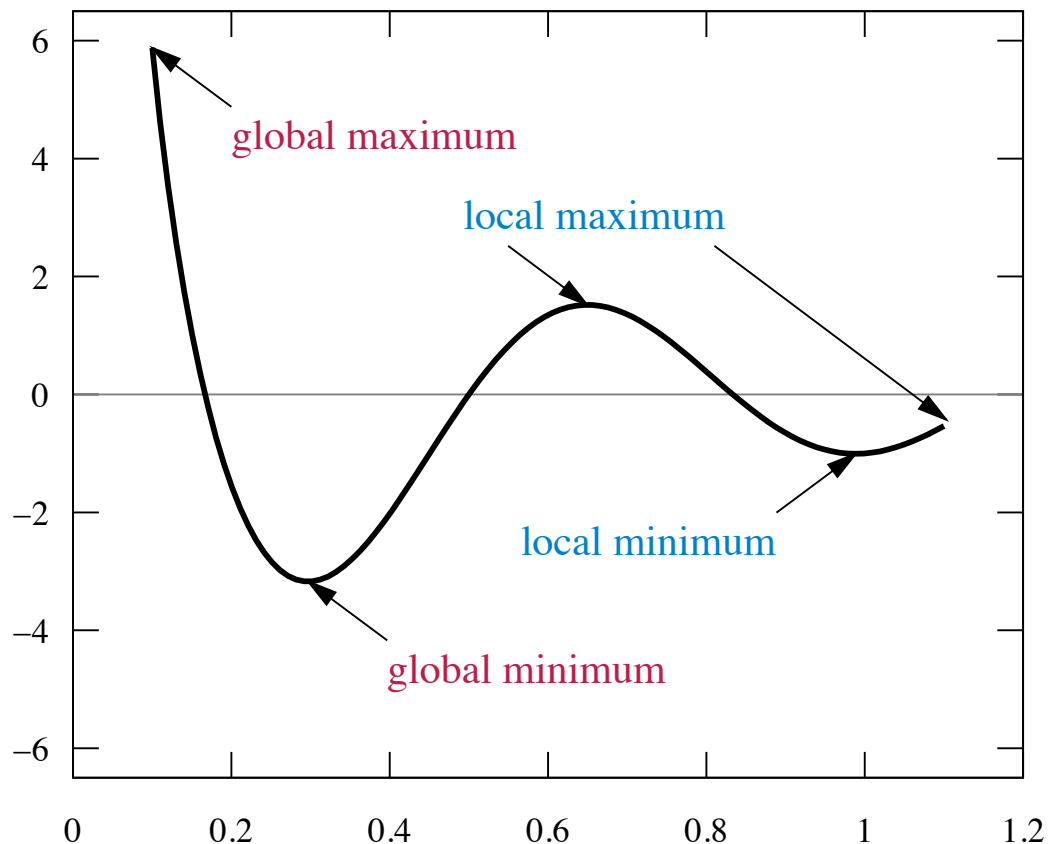
Local vs. global

- A function is **convex** if for any pair of points on the graph of the function, the line connecting these two points lies above or on the graph.
 - Every local minimum of a convex function is a **global minimum**.
 - A convex function can have more than one global minimizer.

- The MSE objective for linear regression is convex.
- For a nonconvex function, gradient descent may approach a local minimum or another stationary point, or it may fail to converge.
- Standard linear regression, logistic regression, and linear SVM objectives are convex in their parameters. Neural network training objectives generally are not.

A function is concave when the line segment between any two points on its graph lies below or on the graph.

Local vs. global



Attribution: commons.wikimedia.org/wiki/File:Extrema_example.svg

Unbounded objective

```
In [26]: # 1. Define the symbolic variable and the function
x = sp.Symbol('x', real=True)
f_expr = 2*x**3 + 4*x**2 - 5*x + 1

# 2. Compute the derivative of f
f_prime_expr = sp.diff(f_expr, x)
```

```

# 3. Convert symbolic expressions to Python functions
f = sp.lambdify(x, f_expr, 'numpy')
f_prime = sp.lambdify(x, f_prime_expr, 'numpy')

# 4. Generate a range of x-values
x_vals = np.linspace(-4, 2, 1000)

# 5. Compute f and f' over this range
y_vals = f(x_vals)
y_prime_vals = f_prime(x_vals)

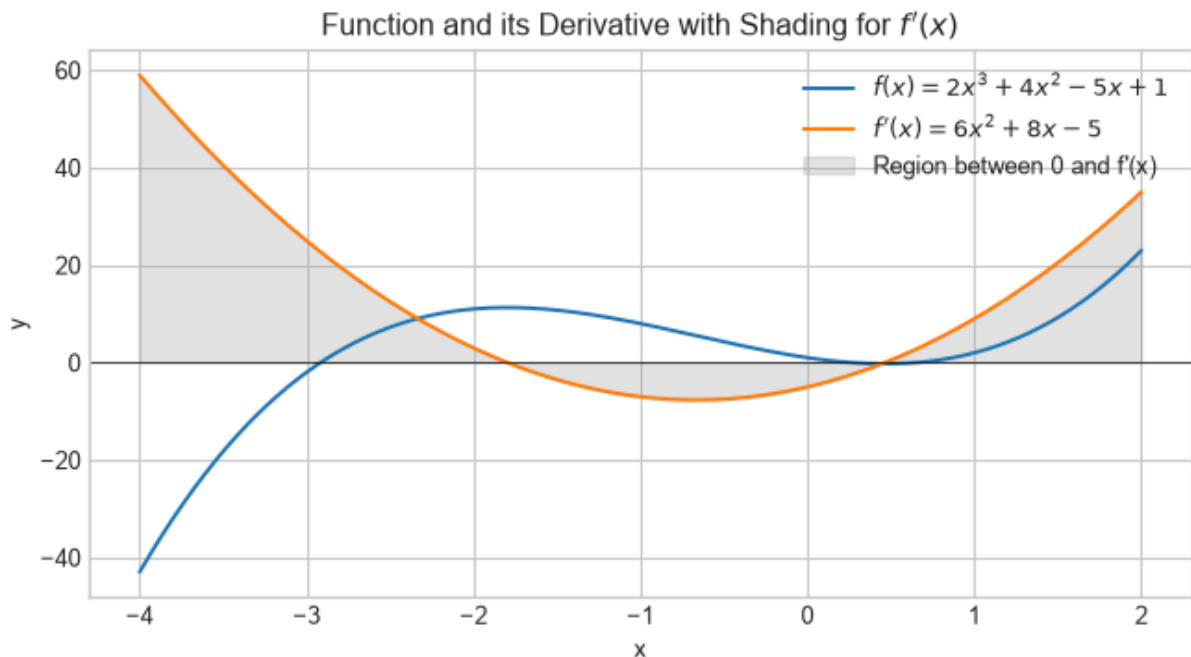
# 6. Prepare LaTeX strings for legend
f_label = rf'$f(x) = {sp.latex(f_expr)}$'
f_prime_label = rf'$f^{prime}(x) = {sp.latex(f_prime_expr)}$'

# 7. Plot f and f', with equations in the legend
plt.figure(figsize=(8, 4))
plt.plot(x_vals, y_vals, label=f_label)
plt.plot(x_vals, y_prime_vals, label=f_prime_label)

# 8. Shade the region between x-axis and f'(x) for the entire domain
plt.fill_between(x_vals, y_prime_vals, 0, color='gray', alpha=0.2, interpolate=True,
                 label='Region between 0 and f\'(x)')

# 9. Add reference line, labels, legend, etc.
plt.axhline(0, color='black', linewidth=0.5)
plt.title(rf'Function and its Derivative with Shading for $f^{prime}(x)$')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()

```

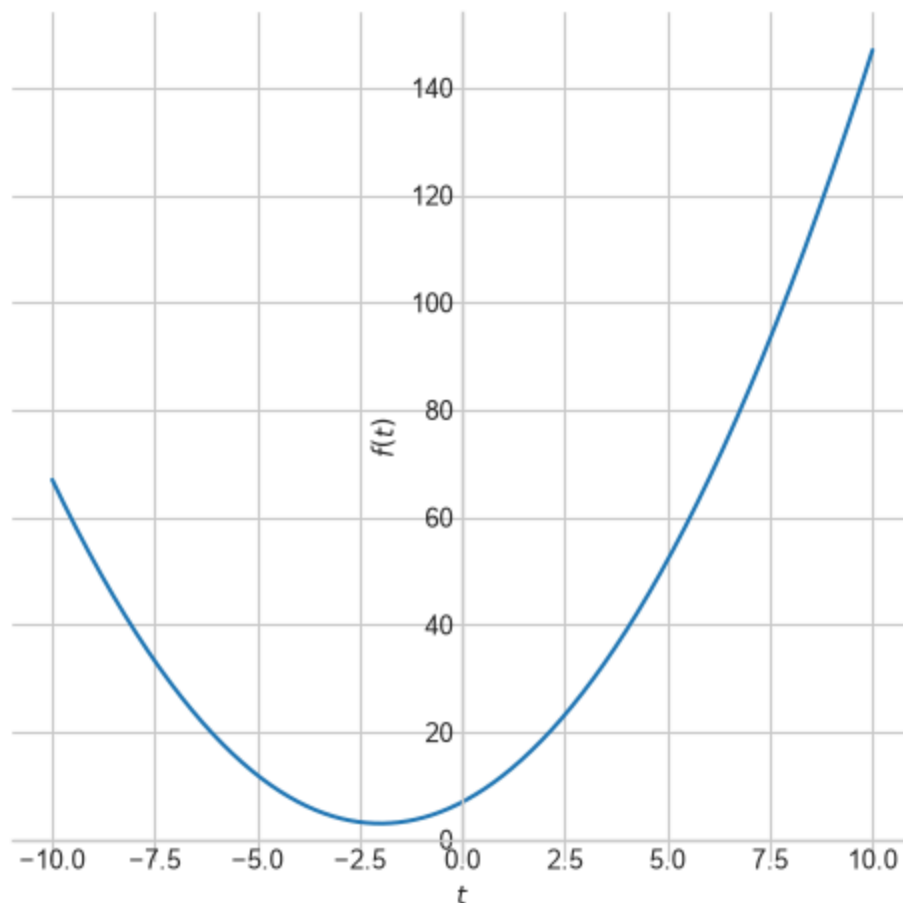


For **functions lacking a global minimum**, gradient descent can **continue descending indefinitely**, preventing convergence.

- This example shows that being differentiable is not sufficient to guarantee convergence. The objective must also have appropriate geometry, and the learning rate must be suitable.
- Because this cubic function is unbounded below, gradient descent can continue decreasing the objective without approaching a finite minimizer.

Learning Rate

```
sp.plot(quadratic_expression, size=(5, 5))
```



- **Small steps**, low values for α , will make the algorithm **converge slowly**.
- **Large steps** might cause the algorithm to **diverge**.
- When the gradient approaches zero near a minimum, fixed-rate updates become smaller.

Learning Rate

```

In [28]: def quadratic_cost(x):
          return x**2

          def quadratic_gradient(x):
              return 2*x

          # Initial guess, learning rate, and number of gradient-descent steps
          parameter_value = 2.0
          learning_rate = 1.1 # Too large => divergence
          num_iterations = 5 # We'll do five updates

          # Store each x value in a list (trajectory) for plotting
          parameter_trajectory = [parameter_value]

          # Perform gradient descent
          for _ in range(num_iterations):
              gradient_value = quadratic_gradient(parameter_value)
              parameter_value -= learning_rate * gradient_value
              parameter_trajectory.append(parameter_value)

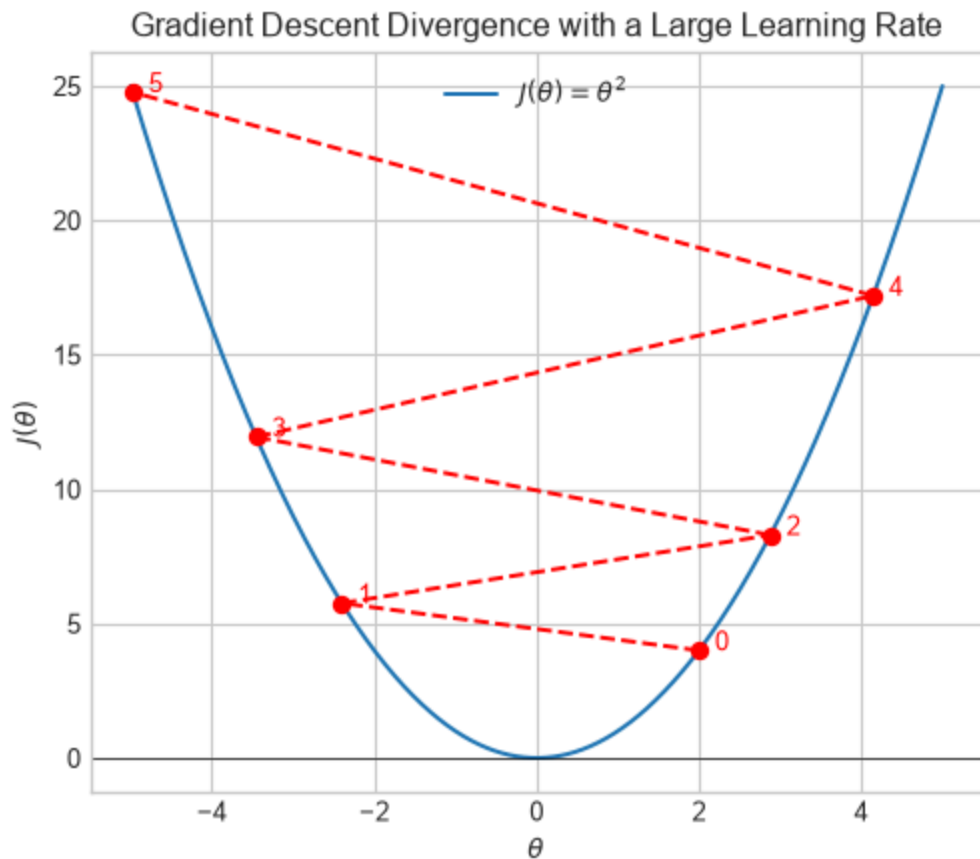
          # Prepare data for plotting
          parameter_values = np.linspace(-5, 5, 1000)
          cost_values = quadratic_cost(parameter_values)

          # Plot the objective.
          plt.figure(figsize=(6, 5))
          plt.plot(parameter_values, cost_values, label=r"$J(\theta) = \theta^2$")
          plt.axhline(0, color='black', linewidth=0.5)

          # Plot the trajectory, labeling each iteration
          for iteration, theta_t in enumerate(parameter_trajectory):
              cost_t = quadratic_cost(theta_t)
              plt.plot(theta_t, cost_t, 'ro')
              plt.text(theta_t, cost_t, f" {iteration}", color='red')
              if iteration > 0:
                  theta_previous = parameter_trajectory[iteration - 1]
                  previous_cost = quadratic_cost(theta_previous)
                  plt.plot([theta_previous, theta_t], [previous_cost, cost_t], 'r--')

          # Final touches
          plt.title("Gradient Descent Divergence with a Large Learning Rate")
          plt.xlabel(r"$\theta$")
          plt.ylabel(r"$J(\theta)$")
          plt.legend()
          plt.grid(True)
          plt.show()

```



Batch gradient descent

- This algorithm is called **batch gradient descent** because every update uses the entire training set.

...

- Features on very different scales can make the objective poorly conditioned and slow convergence.

Batch gradient descent - drawback

- Each batch update becomes more expensive as the **number of training examples increases**.

...

- Every update processes **all** training examples, which can make batch gradient descent impractical for data that do not fit in memory.

Stochastic Gradient Descent

Stochastic gradient descent uses **one** training example for each parameter update. The examples are normally shuffled before each epoch.

```
epochs = 10
rng = np.random.default_rng(42)
for epoch in range(epochs):
    for selection in rng.permutation(N):
        # Calculate the gradient using one example.
        # Update the parameters.
```

- Each update is inexpensive, which makes SGD suitable for large or streaming datasets.
- Its trajectory is noisier than that of batch gradient descent.
 - On a nonconvex objective, this noise can help move away from some saddle points or shallow local minima.
 - The noise does not guarantee convergence to a global minimum.

Progressively reducing the learning rate can limit oscillation near a minimum. This technique is called a **learning schedule**.

Random selection or shuffling reduces systematic bias caused by the order of the examples, but it does not guarantee a global optimum.

Mini-batch size

A **mini-batch** estimates the gradient from a random subset of the training set.

```
In [29]: # Use one shuffled sequence so that each larger mini-batch contains the
# examples highlighted in the preceding panel.
rng = np.random.default_rng(42)
shuffled_indices = rng.permutation(len(faithful_y_train))

batch_sizes = [1, 4, 16, len(faithful_y_train)]
batch_colours = ["#0072B2", "#E69F00", "#009E73", "#CC79A7"]
batch_titles = [
    r"$B=1$ (stochastic)",
    r"$B=4$",
    r"$B=16$",
    rf"$B=N={len(faithful_y_train)}$ (batch)",
]

fig, axes = plt.subplots(2, 2, figsize=(10, 5), sharex=True, sharey=True)
x_train = faithful_X_train[:, 0]

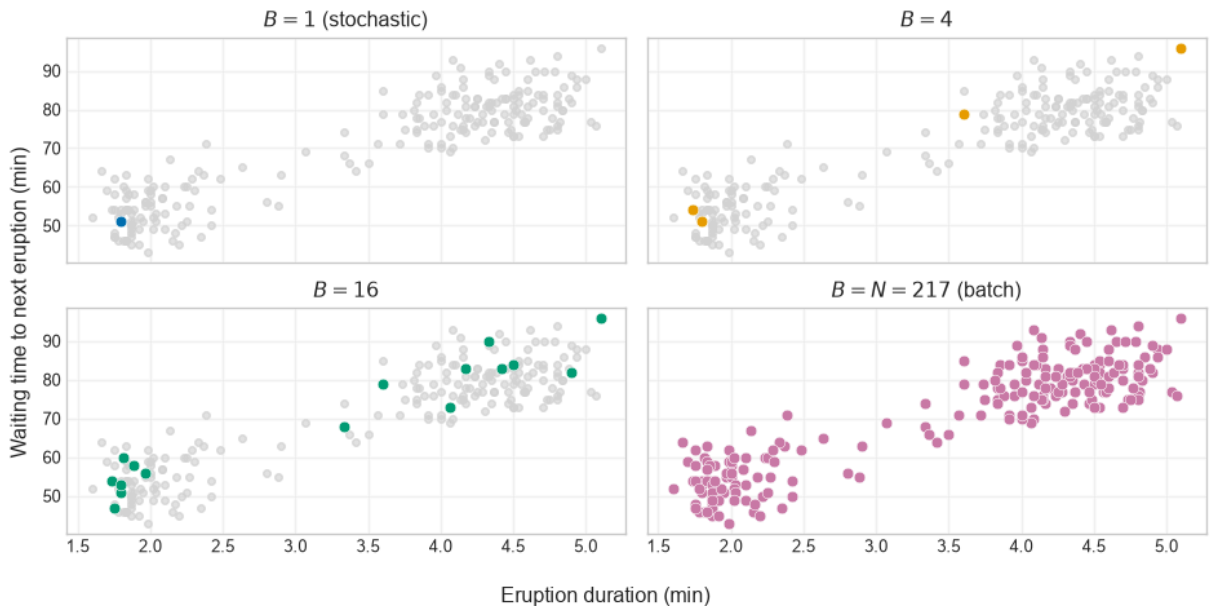
for ax, batch_size, colour, title in zip(
    axes.flat, batch_sizes, batch_colours, batch_titles
):
    selected = shuffled_indices[:batch_size]
    ax.scatter(x_train, faithful_y_train, color="lightgray", s=18, alpha=0.6)
    ax.scatter(
```

```

x_train[selected],
faithful_y_train[selected],
color=colour,
edgecolor="white",
linewidth=0.4,
s=42,
)
ax.set_title(title)
ax.grid(alpha=0.2)

fig.supxlabel("Eruption duration (min)")
fig.supylabel("Waiting time to next eruption (min)")
plt.tight_layout()
plt.show()

```



As B increases, the mini-batch gradient generally becomes **less variable**; when $B = N$, it equals the **batch gradient**.

The light-grey points show the entire training set. The highlighted selections are nested: each larger mini-batch contains all the examples highlighted in the preceding panel. This isolates the effect of adding examples rather than comparing unrelated random selections.

The panels show which data are used to compute each gradient estimate; they do not display the gradient itself. The gradient is a vector in parameter space. For fixed parameter values, the mini-batch gradient is the average of the individual-example gradients over the highlighted examples.

A small random mini-batch can reflect the structure of the training set, but this is not guaranteed for every draw. Increasing B reduces the variability of the gradient estimate on average; an individual larger mini-batch need not be closer to the full-data gradient than every smaller one. When $B = N$, the mini-batch and batch gradients are identical.

Mini-batches can also take advantage of hardware acceleration for matrix operations, particularly on GPUs.

Larger batches are not always better. A smaller batch uses less of the training set, so its gradient may point in a somewhat different direction from the gradient computed using all the examples. The direction can therefore change more noticeably from one mini-batch to the next.

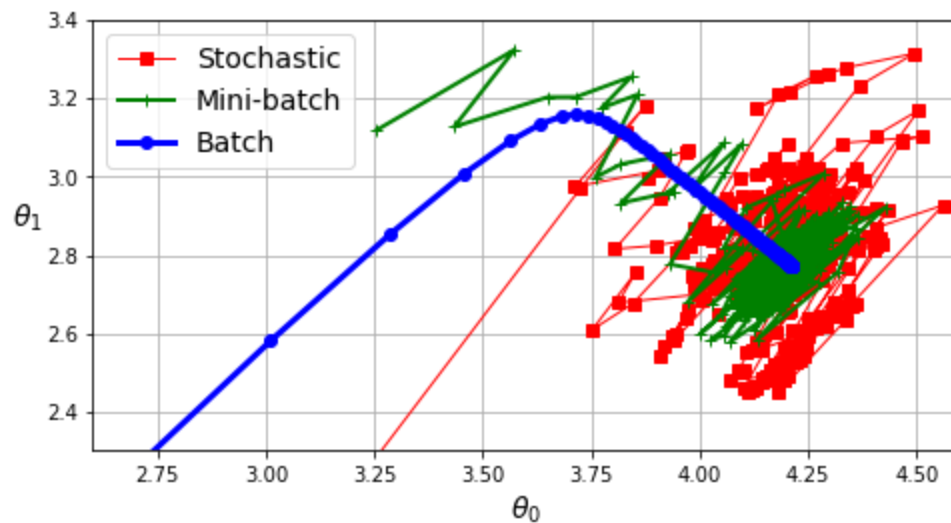
There are two distinct ideas worth separating:

1. **Optimization:** The batch gradient averages the contributions of every training example. A mini-batch gradient averages only the selected examples, so it is an approximation. As the batch size increases, the direction generally becomes more stable and more similar to the batch-gradient direction. When $B = N$, the two are identical. For non-convex objectives, such as those used to train deep networks, the variation between updates can sometimes help the optimizer move away from saddle points. This benefit is not guaranteed. It does not apply in the same way to the convex MSE objective used here for linear regression, in which every local minimum is global.
2. **Generalization:** In some deep-learning experiments, models trained with smaller batches have performed better on new data. This is not a general rule: with a well-adjusted learning rate, training schedule, and training duration, large batches can perform just as well. The randomness of mini-batch training can influence which solution is found, even when no explicit penalty is added; this effect is often called **implicit regularization**.

Why not use the entire training set to compute every gradient?

A mini-batch is cheaper per update than using the complete training set and can be processed efficiently using matrix operations. Its gradient is usually accurate enough to make useful progress, and its randomness is not necessarily harmful. Depending on the problem and training procedure, it can sometimes help optimization or generalization.

Stochastic, Mini-Batch, Batch



Attribution: Géron (2022), Figure 4.10, [04_training_linear_models.ipynb](#)

Summary

- **Batch gradient descent** computes an exact full-data gradient, but each update can be expensive for a large dataset.
- **Stochastic gradient descent** uses inexpensive, noisy updates and can process examples incrementally.
- **Mini-batch gradient descent** balances gradient noise with efficient matrix operations and hardware acceleration.

The typical size of a mini-batch when applying stochastic gradient descent (SGD) can vary depending on the specific application and dataset, but common sizes often range between 32 and 512 samples. Here are some common mini-batch sizes used in practice:

1. **Small Mini-Batches:** Sizes such as 16, 32, or 64 are often used when working with smaller datasets or when memory constraints are a concern.
2. **Medium Mini-Batches:** Sizes like 128, 256, or 512 are commonly used and can provide a good balance between computational efficiency and convergence speed.
3. **Large Mini-Batches:** Sizes like 1024, 2048, or larger might be used in large-scale machine learning tasks, especially when sufficient computational resources are available.

The choice of mini-batch size can influence several factors such as:

- **Training Speed:** Larger mini-batches can make better use of parallel processing capabilities, potentially speeding up training.
- **Convergence:** Smaller mini-batches can introduce more noise in the gradient estimation, which can sometimes help escape local minima and improve

generalization.

- **Memory Usage:** Larger mini-batches require more memory, which might be a limiting factor, especially on GPUs with limited VRAM.

Ultimately, the optimal mini-batch size is task-specific and often determined empirically through experimentation.

`SGDRegressor` performs stochastic updates. Batch and mini-batch variants require explicit control over how each gradient is aggregated.

Optimization and deep nets

We will briefly revisit the subject when discussing **deep artificial neural networks**, for which **specialized optimization algorithms** exist.

- Momentum Optimization
- Nesterov Accelerated Gradient
- AdaGrad
- RMSProp
- Adam and Nadam

Final word

- Optimization is a vast subject. Other algorithms exist and are used in other contexts.
 - Including:
 - Particle swarm optimization (PSO), genetic algorithms (GAs), and artificial bee colony (ABC) algorithms.

Prologue

Linear regression - summary

- In a **regression task**, the model predicts a real-valued label.
- A parameter vector θ is a point in **parameter space**. Each value of θ determines a function h_θ in the **hypothesis space**.
- The **mean squared error** $J(\theta)$ evaluates a parameter choice on a fixed dataset. Training seeks parameter values with lower objective values.

Linear regression - summary (contd)

- Gradient descent updates all parameters using

$$\theta \leftarrow \theta - \alpha \nabla J(\theta),$$

where the learning rate α controls the step size.

- For linear regression, the MSE objective is **convex**, so every local minimum is also a global minimum.
- **Batch**, **stochastic**, and **mini-batch gradient descent** differ in how many training examples contribute to each update.

Andrew Ng



Machine Learning

Linear regression
with one variable

Gradient descent

- [Gradient Descent \(Math\)](#)
(11:30 m)
- [Intuition](#)
(11:51 m)
- [Linear Regression](#)
(10:20 m)
- [ML-005 | Stanford | Andrew Ng](#)
(19 videos)

[Andrew Ng](#) is presenting the gradient descent algorithm using a linear regression with one variable.

Andrew Ng is Founder of [DeepLearning.AI](#), Founder & CEO of [Landing AI](#), General Partner at [AI Fund](#), Chairman and Co-Founder of [Coursera](#) and an Adjunct Professor at Stanford University's Computer Science Department.

Ng was also a cofounder and head of [Google Brain](#) and was the former Chief Scientist at [Baidu](#).

Fundamentals by Herman Kamper

[https://www.youtube.com/watch?](https://www.youtube.com/watch?v=BlnLoqn3ZBo&list=PLmZIBlcArwhOr0ysO1Hg4Wfoww0dZnHz4)

[v=BlnLoqn3ZBo&list=PLmZIBlcArwhOr0ysO1Hg4Wfoww0dZnHz4](https://www.youtube.com/watch?v=BlnLoqn3ZBo&list=PLmZIBlcArwhOr0ysO1Hg4Wfoww0dZnHz4)

Mathematics

3Blue1Brown

- [Essence of linear algebra](#)
 - A series of 16 videos (10 to 15 minutes per video) providing “a geometric understanding of matrices, determinants, eigen-stuffs and more.”
 - **64.7 million** views as of September 18, 2026.
- [Essence of calculus](#)
 - A series of 12 videos (15 to 20 minutes per video): “The goal here is to make calculus feel like something that you yourself could have discovered.”
 - **45.407 million** views as of September 18, 2026.

Next lecture

- Logistic regression

Appendix

LinearRegression

```
In [30]: np.random.seed(42)

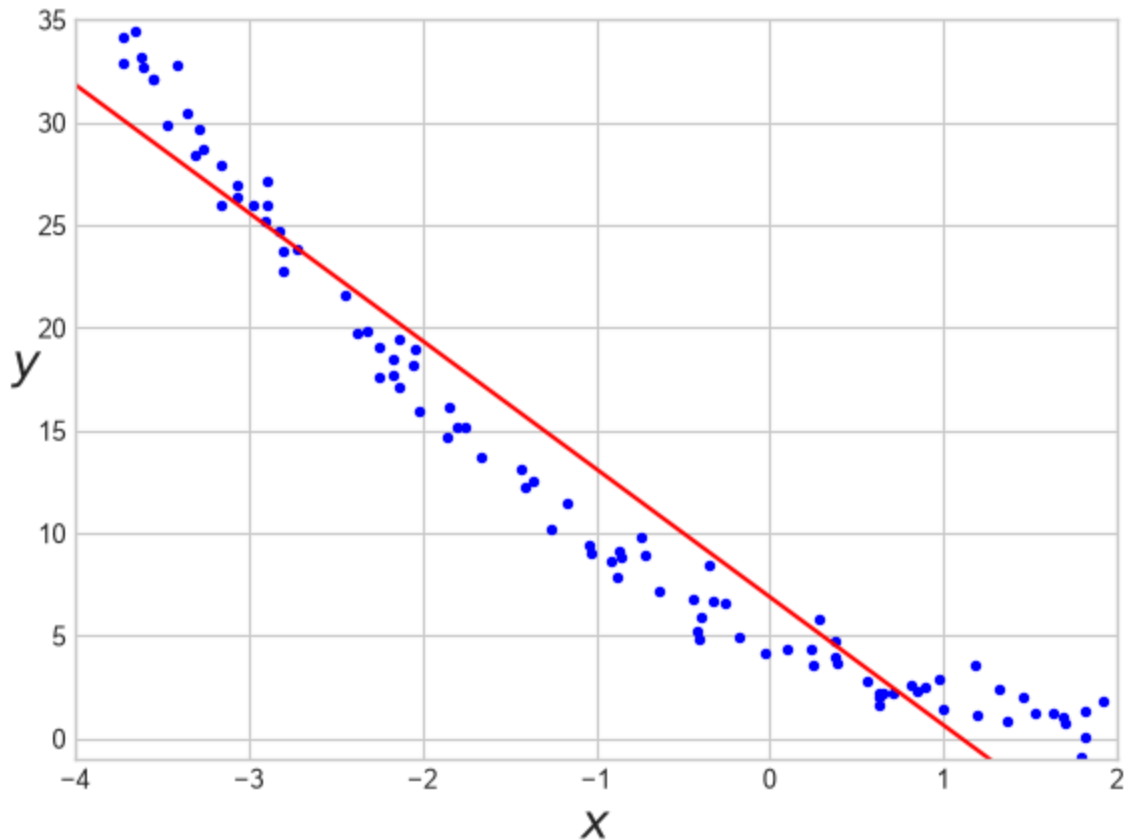
X = 6 * np.random.rand(100, 1) - 4
y = X ** 2 - 4 * X + 5 + np.random.randn(100, 1)

lin_reg = LinearRegression()
lin_reg.fit(X, y)

X_new = np.array([[-4], [2]])
y_pred = lin_reg.predict(X_new)

plt.plot(X, y, "b.")
plt.plot(X_new, y_pred, "r-")
plt.xlabel("$x$", fontsize=18)
```

```
plt.ylabel("$y$", rotation=0, fontsize=18)
plt.axis([-4, 2, -1, 35])
# save_fig("regression_linear-01")
plt.show()
```



```
In [31]: lin_reg = LinearRegression()
lin_reg.fit(X, y)
```

Herein, we examine a straightforward model involving a single variable.

Source code

```
In [32]: import numpy as np
from sklearn.linear_model import LinearRegression

np.random.seed(42)
X = 6 * np.random.rand(100, 1) - 4
y = X ** 2 - 4 * X + 5 + np.random.randn(100, 1)

lin_reg = LinearRegression()
lin_reg.fit(X, y)

X_new = np.array([[-4], [2]])
y_pred = lin_reg.predict(X_new)
```

This slide contains an anomaly. Can you identify it?

The data have been generated from a quadratic model, $x^2 - 4x + 5$, with Gaussian noise added to make the synthetic data more realistic.

This example illustrates that, in practice, we often have limited knowledge about the underlying model that generated the data.

Despite this uncertainty, we make assumptions—in this case, assuming linearity—when building our models.

Later, we will refer to this as **bias** (distinct from the bias parameter in our regression model). Both bias and variance influence the performance of our models.

GPT 4o interactions (2024-09-14)

User: Explain the code on the previous slide line by line.

...

AI: Certainly! Here is a line-by-line explanation of the provided code:

```
In [33]: import numpy as np
```

- **Explanation:** This line imports the NumPy library, which is a fundamental package for numerical computations in Python. It is commonly used for working with arrays and matrices, as well as for performing a variety of mathematical operations.

```
In [34]: np.random.seed(42)
```

- **Explanation:** This line fixes NumPy's random seed so that the synthetic dataset is reproducible.

```
In [35]: X = 6 * np.random.rand(100, 1) - 4
```

- **Explanation:** This line generates a NumPy array `X` containing 100 rows and 1 column of random numbers. The `np.random.rand(100, 1)` function generates uniformly distributed random numbers between 0 and 1. Multiplying by 6 scales these numbers to the range [0, 6), and subtracting 4 shifts the range to [-4, 2).

```
In [36]: y = X ** 2 - 4 * X + 5 + np.random.randn(100, 1)
```

- **Explanation:** This line generates the target variable `y` based on a quadratic equation with some added noise. Specifically:
 - `X ** 2` computes the square of each element in `X`.
 - `- 4 * X` subtracts four times each element in `X`.

- `+ 5` adds a constant term of 5.
- `+ np.random.randn(100, 1)` adds Gaussian noise with a mean of 0 and standard deviation of 1 to each element, making the data more realistic by introducing some randomness.

```
In [37]: from sklearn.linear_model import LinearRegression
```

- **Explanation:** This line imports the `LinearRegression` class from the `sklearn.linear_model` module, which is part of the Scikit-Learn library. Scikit-Learn is widely used for machine learning in Python.

```
In [38]: lin_reg = LinearRegression()
```

- **Explanation:** This line creates an instance of the `LinearRegression` class and assigns it to the variable `lin_reg`. This object will be used to fit the linear regression model to the data.

```
In [39]: lin_reg.fit(X, y)
```

- **Explanation:** This line fits the linear regression model to the data by finding the best-fitting line through the points `(X, y)`. The `fit` method trains the model using the provided features `X` and target variable `y`.

```
In [40]: X_new = np.array([[-4], [2]])
```

- **Explanation:** This line creates a new NumPy array `X_new` with two values: -4 and 2. These values represent new data points for which we want to predict the target variable using the trained model.

```
In [41]: y_pred = lin_reg.predict(X_new)
```

- **Explanation:** This line uses the trained linear regression model to predict the target variable `y_pred` for the new data points in `X_new`. The `predict` method returns the predicted values based on the fitted model.

In summary, this script generates synthetic data, fits a linear regression model to it, and then uses the model to make predictions on new data points.

Source code (continued)

```
In [42]: import os
```

```
def save_fig(fig_id, tight_layout=True, fig_extension="pdf", resolution=300):
    path = os.path.join(fig_id + "." + fig_extension)
    print("Saving figure", fig_id)
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

Source code (continued)

```
In [43]: plt.plot(X, y, "b.")
plt.plot(X_new, y_pred, "r-")
plt.xlabel("$x$", fontsize=18)
plt.ylabel("$y$", rotation=0, fontsize=18)
plt.axis([-4, 2, -1, 35])
save_fig("regression_linear-01")
plt.show()
```

References

Azzalini, A., and A. W. Bowman. 1990. "A Look at Some Data on the Old Faithful Geyser." *Journal of the Royal Statistical Society Series C: Applied Statistics* 39 (3): 357–65. <https://doi.org/10.2307/2347385>.

Géron, Aurélien. 2022. *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd ed. O'Reilly Media, Inc.

Russell, Stuart, and Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson. <http://aima.cs.berkeley.edu/>.

Stanton, Jeffrey M. 2001. "Galton, Pearson, and the Peas: A Brief History of Linear Regression for Statistics Instructors." *Journal of Statistics Education* 9 (3). <https://doi.org/10.1080/10691898.2001.11910537>.

Marcel **Turcotte**

Marcel.Turcotte@uOttawa.ca

School of Electrical Engineering and **Computer Science** (EECS)

University of Ottawa